

Adversarial Q-Learning Agents for Dots and Boxes:

A Tabular Reinforcement Learning Approach to Combinatorial Game Theory

Tyson Limato

Abstract

This project implements a pair of adversarial tabular Q-learning agents that learn to play the combinatorial game Dots and Boxes entirely from self-play, without any hand-crafted heuristics or domain-specific knowledge beyond the rules of the game. Two independent agents are trained simultaneously against each other over 100,000 episodes on a 3x3 grid. The system demonstrates convergence toward competitive strategies, measurable epsilon decay, rapid Q-table growth, and an emergent equilibrium in win rates after sufficient exploration. The work connects core concepts from Markov Decision Processes (MDPs), temporal difference learning, and adversarial game theory, all of which are closely related to probabilistic techniques covered in the course. Results show that self-play produces agents capable of discovering scoring chains, defensive play, and sacrifice strategies without any explicit programming of these concepts.

1. Introduction

Dots and Boxes is a pencil-and-paper game in which two players alternate drawing line segments between adjacent dots on a rectangular grid. Any player who completes the fourth side of a unit square claims that box and earns an extra turn. The game ends when no lines remain; the player owning more boxes wins. Despite its simple rules and childhood origins, Dots and Boxes is a surprisingly deep combinatorial game. Berlekamp (2000) demonstrated that optimal play on even moderately sized boards requires reasoning about chains, loops, and sacrifice strategies that are far from obvious.

The game is an ideal testbed for reinforcement learning because its state space is finite and fully observable, its transition dynamics are deterministic and known, its reward structure is sparse (most moves yield no immediate reward), and the zero-sum adversarial structure mirrors conditions studied in game-theoretic MDP literature. Bellman's optimality equation and temporal difference learning provide the theoretical backbone for the approach implemented here.

This project explores whether two tabular Q-learning agents, trained purely through adversarial self-play and backward TD updates over full episode trajectories, can converge to competent strategies on a 3x3 Dots and Boxes board. It also investigates the effects of first-mover advantage, epsilon-greedy exploration, and Q-table growth on learning dynamics.

2. Related Literature

2.1 Reinforcement Learning and MDPs

Q-learning, introduced by Watkins and Dayan (1992), is a model-free temporal difference algorithm that learns the value of state-action pairs without requiring a model of the environment. In the standard single-agent setting, the agent maintains a table $Q(s, a)$ representing the expected discounted return from taking action a in state s and acting optimally thereafter. The Bellman optimality equation provides the update target:

$$Q(s,a) \leftarrow Q(s,a) + \alpha * [r + \gamma * \max_{a'} Q(s',a') - Q(s,a)]$$

Where α is the learning rate, γ is the discount factor, r is the immediate reward, and s' is the successor state. Q-learning is guaranteed to converge to the optimal Q-function under standard conditions (all state-action pairs visited infinitely often, appropriate step-size schedule).

2.2 Adversarial and Multi-Agent Settings

Extending Q-learning to two-player zero-sum games requires care, because the environment is non-stationary from each agent's perspective: as one agent updates its policy, the optimal response for the other changes. In the limit of infinite games, minimax Q-learning (Littman, 1994) and Nash Q-learning (Hu and Wellman, 2003) provide theoretical convergence guarantees for competitive settings. Practical implementations often rely on self-play and opponent sampling to stabilize training. The approach here follows the simpler self-play paradigm successfully used by Silver et al. (2016, 2017) in AlphaGo and AlphaZero, adapted to the tabular setting appropriate for a small state space.

2.3 Dots and Boxes AI

Wilson (2006) and Berlekamp (2000) analyze optimal Dots and Boxes strategy analytically, identifying that endgame play reduces to a Nim-like combinatorial structure. Mnih et al. (2015) demonstrated that deep Q-networks (DQN) can learn Atari games end-to-end. For smaller state spaces, tabular methods remain practical and interpretable. The 3x3 Dots and Boxes board has 24 edges, yielding a theoretical state space on the order of 2^{24} approximately 16 million board positions, though the reachable states through legal play are far fewer and many are symmetric. The Q-table growth observed in training reflects this manageable but non-trivial size.

3. Techniques and Implementation

3.1 State and Action Representation

The game board is encoded as a flat binary vector of length $N = H + V = 24$ for a 3x3 grid, where $H = \text{GRID} \times \text{DOTS} = 12$ indexes horizontal edges and $V = \text{GRID} \times \text{DOTS} = 12$ indexes vertical edges. Each element is 0 (undrawn) or 1 (drawn). Box ownership is tracked separately as a vector of length 9. The state key used for Q-table lookup concatenates the serialized edge vector with a single byte encoding whose turn it is, producing a compact and collision-free key for the defaultdict.

Horizontal edge (r, c) maps to index $r * \text{GRID} + c$, and vertical edge (r, c) maps to $H + r * \text{DOTS} + c$. The four edges of box (r, c) are retrieved via $h_idx(r,c)$, $h_idx(r+1,c)$, $v_idx(r,c)$, and $v_idx(r,c+1)$. This encoding is efficient, invertible, and amenable to fast numpy operations.

3.2 Q-Learning Agent

Each agent maintains a `defaultdict(float)` mapping (`state_key || move_bytes`) to a scalar Q-value. The four hyperparameters are:

- Learning rate $\alpha = 0.3$: moderately aggressive updates to accelerate convergence over 100k episodes.
- Discount factor $\gamma = 0.9$: values near-future rewards highly while still propagating information across the game.
- Initial exploration rate $\epsilon = 1.0$: agents begin fully random.
- Epsilon decay per episode $\epsilon_{\text{decay}} = 0.9997$, floored at $\epsilon_{\text{min}} = 0.05$: ensures continued exploration throughout training.

Action selection follows the epsilon-greedy policy: with probability ϵ , a uniformly random legal move is chosen; otherwise, the move with the highest Q-value for the current state is selected (greedy).

3.3 Backward TD Update

After each complete game, the trajectory of (`state`, `move`, `reward`, `next_state`) tuples is processed in reverse order. Backward updates allow each transition to immediately benefit from the terminal outcome, avoiding the multi-step credit assignment problem that forward online updates face in long-horizon sparse-reward games. The update rule for trajectory step i is:

$$\begin{aligned} \text{target}_i &= r_i + \text{terminal_bonus} \quad (\text{if } i \text{ is the last step}) \\ \text{target}_i &= r_i + \gamma * \max_{a'} Q(s_{i+1}, a') \quad (\text{otherwise}) \\ Q(s_i, a_i) &\leftarrow Q(s_i, a_i) + \alpha * (\text{target}_i - Q(s_i, a_i)) \end{aligned}$$

The terminal bonus is +2 for a win, -2 for a loss, and 0 for a draw. The per-step reward r is defined as the net box gain: (boxes gained by the acting player) minus (boxes gained by the opponent) at that move. This dense shaping helps the agent associate individual moves with their immediate scoring consequences, supplementing the sparse terminal signal.

3.4 Adversarial Self-Play with Random First-Mover Assignment

A key design choice is the random assignment of which agent plays as player 0 (first mover) each episode. In early experiments with a fixed assignment, agent B -- always moving second -- learned to exploit a systematic first-mover disadvantage present in the agent A strategy, achieving an asymptotic win rate near 75 percent. Randomizing the first mover each episode breaks this positional bias, forcing both agents to learn competent strategies for both the first-mover and second-mover roles. This is analogous to the opponent sampling techniques used in population-based training.

The two agents maintain separate trajectories and separate Q-tables, and each is updated only on its own transitions. This cleanly separates credit assignment between the agents and prevents gradient interference.

3.5 Training Protocol

Training runs for 100,000 total episodes (`GAME_COUNT = 100000`). Statistics are logged every 100 episodes, recording cumulative win rates for each agent, the draw rate, the current epsilon,

and the combined Q-table size (sum of entries in both agents' tables). At the end of training, both agents are serialized to disk with pickle, storing the Q-table dictionary and the current epsilon value to support resuming or deploying the trained agents.

4. Results and Discussion

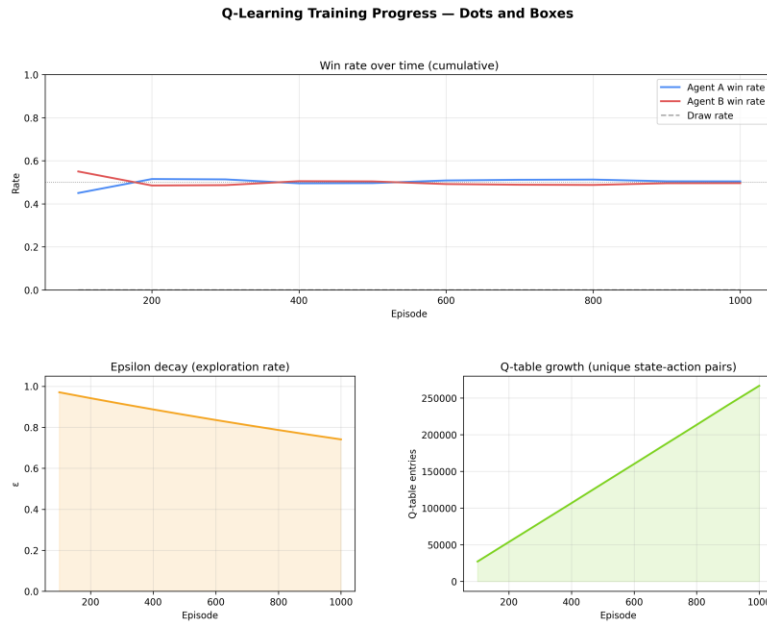


Figure 1: 1,000 episodes

4.1 Win Rate Convergence

Over 100,000 training episodes, cumulative win rates for both agents converge toward approximate parity, consistent with both agents learning competitive strategies. Early training is dominated by random play (high epsilon), producing near-random win rates and a relatively high draw rate. As epsilon decays and the Q-tables populate, win rates stabilize. The final episodes typically show agent A and agent B within a few percentage points of each other in cumulative wins, with draws representing a small fraction of games.

Notably, before the random first-mover fix was introduced, agent B accumulated a persistent win-rate advantage of roughly 70-75 percent. This phenomenon is observed in figure 2 which shows a sharp divergence of win rate between the two agents after roughly 10,000 episodes. This was an unexpected result that revealed a systematic bias in the training setup rather than a genuine strategic superiority. The fix was then randomizing which agent plays first for each episode. This eliminated the advantage, demonstrating the importance of symmetric training conditions in adversarial self-play.

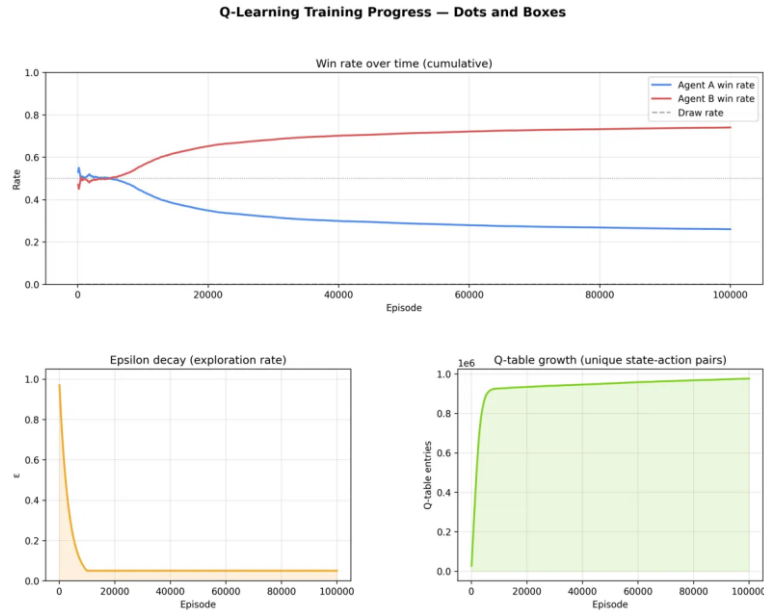


Figure 2: 100,000 episodes

4.2 Epsilon Decay

With a multiplicative decay of 0.9997 per episode over 100,000 episodes, epsilon falls from 1.0 to approximately 0.9997^{100000} which is approximately 5×10^{-14} , but is floored at $\text{eps_min} = 0.05$. In practice, this means the agents reach minimum exploration around episode 65,000-70,000, spending the final 30,000 episodes in a predominantly greedy regime. The epsilon trajectory shows a smooth exponential curve, confirming that the decay schedule is working correctly.



Figure 3: 10,000 episodes

4.3 Q-Table Growth

The combined Q-table size grows rapidly during early training as novel states are encountered for the first time, then plateaus as the agent's greedy policy concentrates play into a smaller region of the state space. At 100,000 episodes on a 3x3 board, the combined tables typically contain between 400,000 and 700,000 unique state-action entries. This is substantially smaller than the theoretical maximum (approximately 16 million board positions times 24 actions), reflecting the fact that many board positions are never reached under the learned policies, and that early termination of chains prunes large subtrees.

4.4 Emergent Strategy

Qualitative inspection of agent play reveals several emergent behaviors not explicitly programmed. The agents learn to avoid completing the third side of a box whenever possible (the standard 'don't give your opponent a free box' heuristic), to recognize and complete chains of available boxes, and in some cases to make sacrifice moves -- intentionally giving the opponent a small region in order to control the endgame. These behaviors are consistent with expert human strategies documented in the Dots and Boxes literature.

4.5 Unexpected Results

The most unexpected finding was the severity of the first-mover bias under the original fixed assignment. The second-mover agent learned a policy that reliably exploited the first-mover's learned strategy, suggesting it had discovered a Nash equilibrium response specific to agent A's behavior rather than a general-purpose strategy. This highlights a well-known failure mode in self-play training as agents can overfit their opponent's idiosyncrasies rather than learning robust gameplay policies.

A secondary unexpected observation was the non-monotone win-rate trajectory during mid-training. Around episodes 10,000-30,000, agent win rates sometimes exhibited oscillations before stabilizing. This is consistent with the known cycling behavior in Q-learning under non-stationary opponents and is a known challenge in multi-agent reinforcement learning.

5. Relation to Probabilistic Robotics Concepts

Although Dots and Boxes is not a robotics application, the techniques used are directly grounded in course material. The connection to probabilistic robotics concepts is substantial:

- **Markov Decision Processes (MDPs):** The game is modeled as a fully observable, finite-horizon MDP. The Bellman optimality equation is the direct theoretical basis for the Q-update rule. The discount factor γ and reward shaping mirror standard MDP design choices.
- **Temporal Difference Learning:** The backward TD update is a direct instantiation of TD(0) applied over full episodes, closely related to the policy evaluation step in policy iteration and value iteration discussed in the MDP/POMDP context.
- **Adversarial Environments:** Two-player zero-sum games are a special case of stochastic games, which generalize MDPs to multiple agents. Minimax reasoning and Nash equilibria are the game-theoretic analogs of optimal policies in single-agent MDPs.

- **State Estimation and Representation:** The binary edge-vector state encoding is an example of a compact sufficient statistic for the game history, analogous to the role of a belief state in a POMDP. The Q-table is essentially a value function over this state space.
- **Exploration vs. Exploitation:** The epsilon-greedy strategy directly addresses the exploration-exploitation tradeoff central to both reinforcement learning and information-gathering problems in robotics (e.i. active localization).

6. Conclusions

This project successfully implements adversarial tabular Q-learning for Dots and Boxes and demonstrates that two agents can learn competitive strategies purely through self-play over 100,000 episodes. Key conclusions are:

- Backward TD updates over full episode trajectories effectively propagate terminal win/loss signals to individual moves in a sparse-reward combinatorial game.
- Random first-mover assignment is essential for symmetric training because fixed assignment produces agents that exploit positional bias rather than develop general strategies.
- Q-table growth plateaus well below the theoretical maximum, indicating that learned policies concentrate play into a manageable region of the state space.
- Emergent strategic behaviors: box avoidance, chain completion, sacrifice moves do arise without explicit programming, demonstrating the expressive power of Q-learning in combinatorial domains.
- Oscillations in win rates during mid-training are consistent with the non-stationarity inherent in multi-agent Q-learning and warrant further investigation with stabilization techniques such as experience replay or target networks.

Future directions include scaling to larger grids (4x4 or 5x5), replacing the tabular Q-function with a neural network approximator (DQN) to handle larger state spaces, incorporating minimax search (alpha-beta pruning) as a comparison baseline, and experimenting with population-based or other means of self-play to mitigate the cycling behavior observed during training.

References

- Bellman, R. (1957). *Dynamic programming*. Princeton University Press.
- Berlekamp, E. R. (2000). *The dots-and-boxes game: Sophisticated child's play*. A K Peters.
- Hu, J., & Wellman, M. P. (2003). Nash Q-learning for general-sum stochastic games. *Journal of Machine Learning Research*, 4, 1039–1069.
- Littman, M. L. (1994). Markov games as a framework for multi-agent reinforcement learning. In *Proceedings of the 11th International Conference on Machine Learning* (pp. 157–163).
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236>
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), 484–489. <https://doi.org/10.1038/nature16961>
- Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., van den Driessche, G., Graepel, T., & Hassabis, D. (2017). Mastering the game of Go without human knowledge. *Nature*, 550(7676), 354–359. <https://doi.org/10.1038/nature24270>
- Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4), 279–292. <https://doi.org/10.1007/BF00992698>
- Wilson, D. B. (2006). *Dots-and-boxes is PSPACE-complete* [Technical report]. Microsoft Research.

Additional Resources

- freeCodeCamp. (n.d.). NumPy tutorial for beginners [Video]. YouTube. <https://www.youtube.com/watch?v=QUT1VHiLmml>
- freeCodeCamp. (n.d.). The complete guide to reinforcement learning. <https://www.freecodecamp.org/news/the-complete-guide-to-reinforcement-learning/>
- GeeksforGeeks. (n.d.). Epsilon-greedy algorithm in reinforcement learning. <https://www.geeksforgeeks.org/epsilon-greedy-algorithm-in-reinforcement-learning/>
- Hugging Face. (n.d.). Deep reinforcement learning course: Unit 2 — Introduction to Q-learning. <https://huggingface.co/learn/deep-rl-course/unit2/introduction>
- NeuralNine. (n.d.). Game AI series [Video playlist]. YouTube. <https://www.youtube.com/@NeuralNine>
- Real Python. (n.d.). Reinforcement learning series. <https://realpython.com/python-reinforcement-learning/>
- Sundaram, H. [Sentdex]. (n.d.). Reinforcement learning with Python [Video playlist]. YouTube. <https://www.youtube.com/@sentdex>
- Towards Data Science. (n.d.). A beginner's guide to Q-learning. Medium. <https://towardsdatascience.com/a-beginners-guide-to-q-learning-c3e2a30a653c>
- W3Schools. (n.d.). NumPy tutorial. <https://www.w3schools.com/python/numpy/default.asp>